PAWSv3 RISC-V CPU Pipeline Architecture

Pipeline Mechanism

The PAWSv3 CPU is a 64-bit decoupled architecture utilizing a 6-stage pipeline, connected by strict ready/acknowledge handshaking signals (READY, ACK) between modules.

- **Stage 1: Fetch (cpu_fetch)**
- Retrieves instructions from the L1 instruction cache.
- Automatically expands Risc-V Compressed Instructions (RVC) into standard 32-bit instructions via dedicated expanders (compressed00, compressed01, compressed10).
- Translates proprietary T-Head (XThead) vendor extensions into native standard operations at fetch time.
- **Stage 2: Decode (cpu_decode)**
- Decodes the instruction utilizing embedded ROM lookup tables to determine floating-point/integer status, necessary ALU operations, and destination registers.
- **Stage 3: Registers (cpu_registers)**
- Manages 32 integer and 32 floating-point registers implemented in dual-port BRAM.
- Responsible for pipeline hazard detection. The unit generates SR_MASK (source) and RD_MASK (destination) bitmasks to compare in-flight register usage against the downstream execution and commit units.
- **Stage 4A: Address Generation (cpu_agu)**
- Computes load/store and jump addresses, evaluates integer branch conditions, and updates the branch predictor.
- Directly dispatches fast-path instructions (LUI, AUIPC, simple jumps) to the Fastbase execution unit, and standard ALU operations to the ALU.
- **Stage 4B: Dispatch (cpu_dispatch)**
- Routes more complex operations to their respective execution units: Integer Division, Control and Status Registers/Floating-Point Unit (CSRFPU), and Load/Store/Atomic (LSA).
- **Stage 5: Execution**
- Processes instructions in parallel across five dedicated logic blocks: ALU, ALUDivide, Fastbase, LSA, and CSRFPU.

- **Stage 6: Commit (cpu_commit_i, cpu_commit_f)**
- Writes results back to the integer and floating-point BRAM registers, 1 integer and 1 floating-point register can be written per clock cycle.
- Write-backs are multiplexed using a strict priority system to prevent collisions. Integer priority is: LSA -> Fastbase -> ALU -> CSRFPU -> Divide. Floating-point priority is: LSA -> CSRFPU.

Pipeline Stalls and Flushes

- **Hazard Stalls (Internal):** If Stage 3 (Registers) detects a Read-After-Write (RAW) or Write-After-Write (WAW) hazard via its register bitmasks, it sets a LOCKED flag, drops its READY signal, and stalls the Decode stage until the downstream execution units commit the required registers.
- **Memory Stalls:** If a Load, Store, or Fetch operation is pending, the CPU relies on MEM_BUSY and MEM_ACK signals. The pipeline stalls in wait states (e.g., STATE_WAIT_ACK) until the memory bus signals completion.
- **Branch Flushes:** Handled exclusively by Stage 4A (AGU). If the AGU evaluates a branch and finds the Fetch stage's prediction was incorrect, it asserts a BP_FLUSH signal. This flag propagates backward to Fetch and Decode, voiding in-flight instructions, and immediately redirects the Program Counter to the correct execution path.

Branch Predictor

- The predictor uses a 64-entry Dual-Port BRAM table.
- Entries are composed of a part-address tag, a 1-bit valid flag, and a 2-bit state prediction (Strongly Not Taken, Weakly Not Taken, Weakly Taken, Strongly Taken).
- Stage 1 (Fetch) queries the table using the current PC to guess the branch direction.
- Stage 4A (AGU) evaluates the actual branch outcome and writes corrections back to the predictor table using a specific 2-bit update matrix (BP_UPDATE).

L1 Instruction Cache

- The CPU features a direct-mapped L1 instruction cache sized at 2048 blocks.
- Each cache line contains a 30-bit instruction, a 1-bit compressed flag, a 1-bit valid flag, and a part-address tag matching the CPU's address width.
- To ensure cache coherency when a new program is loaded, a CPU_FENCE signal triggers a hardware state machine that iterates through and invalidates all cache blocks.

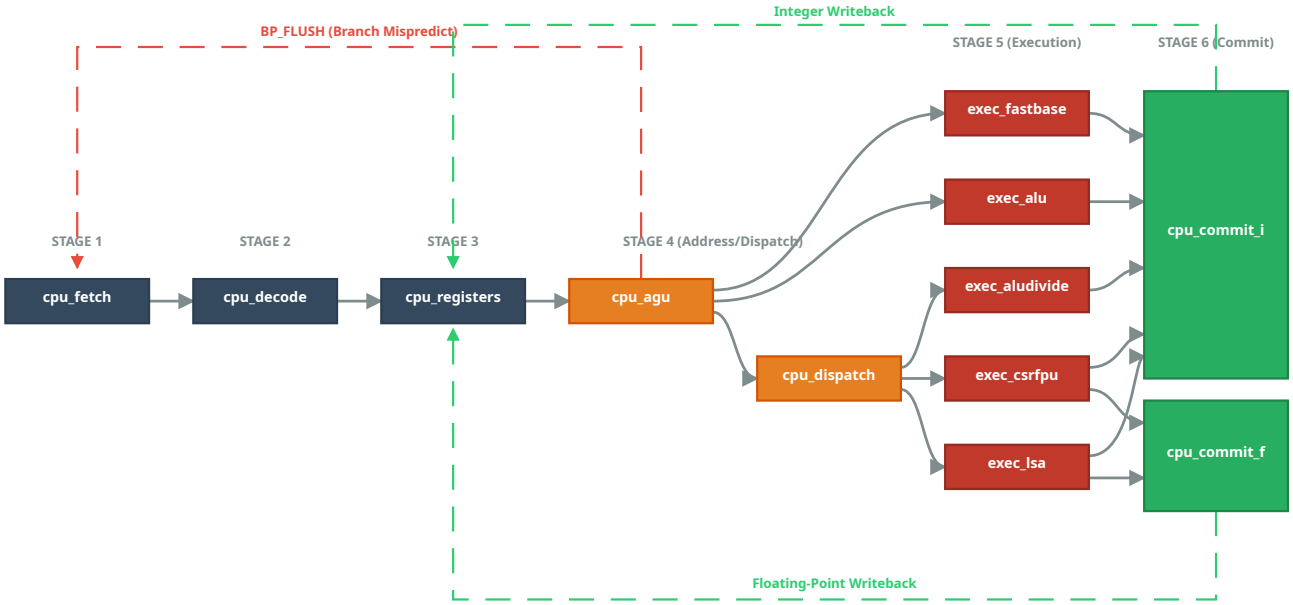
Memory Bus Integration

- The CPU communicates over a 64-bit internal bus linked to a 32-bit external memory controller.
- The memory bus natively handles misaligned operations down to an 8-bit boundary without trapping to software.
- It operates using an iterative state machine (STATE_LOAD_MA, STATE_STORE_MA) that continuously requests subsequent bytes until the total requested payload size (up to 64 bits) is processed.

Instruction Set & Additional Features

- **Radix-8 Mathematics:** To balance logic scale and execution speed, the CPU utilizes highly optimized Radix-8 (3 bits per cycle) algorithms for integer division (dointegerdivide_radix8), floating-point division (do_float_divide_radix8), and floating-point square roots (do_float_sqrt_radix8).
- **Hardware FPU:** Full hardware floating-point support implementing IEEE 754 standards. It natively handles NaN-boxing of Single/Half precision values into Double precision formats internally, features a Fused-Multiply-Add (FMA) unit, and treats subnormal inputs as zero.
- **Atomic Extensions (AMO):** The Load/Store/Atomic (LSA) unit handles atomic memory operations in hardware using a dedicated ALU (aluA), accelerating atomic adds, swaps, XORs, ORs, ANDs, and min/max operations.
- **XThead Extensions:** The pipeline natively supports T-Head vendor extensions (e.g., XTheadBb, XTheadMemPair, XTheadCondMov). Rather than utilizing a separate execution unit, the Fetch module dynamically translates these extensions into native RISC-V instructions.

- **Background Co-processors:** The SoC features a "NuCU" Copper background processor capable of executing independent video/system routines from 1024 program blocks and 64 memory registers.
- **Audio Multiplexing:** Incorporates a hardware audio channel multiplexer utilizing a 7-bit timer for high-frequency 4-bit sample processing.



1. Fetch-Stage Micro-Sequencer (custom_multi)

The instruction fetch stage (cpu_fetch) contains an internal micro-instruction sequence generator (STATE_FETCH_MULTI_2 / MULTI_STEP) that decomposes complex instructions into two back-to-back pipeline operations without requiring memory re-fetches:

- **Memory Pair Instructions (th.lwd, th.lwud, th.ldd, th.swd, th.sdd):** A single instruction in the instruction stream expands into two discrete load or store operations to consecutive aligned registers (rd and $rs2$ / $rd2$) with offset adjustments (+4 or +8 bytes).
- **Auto-Increment Indexed Load/Store (xtheadmemidx):** Pre-increment and post-increment load/store instructions are split into two operations:

1. The primary memory load or store transaction via the Load/Store/Atomic (LSA) pipeline.
2. An automatically synthesized ADDI operation to update the base address register ($rs1$) by the immediate shift index.

2. T-Head Extension Support (XThead)

The CPU supports a substantial subset of T-Head vendor extensions using a hybrid approach of early fetch-stage rewriting and specialized execution paths:

Extension	Instructions	Implementation Mechanism
XTheadBa	th.addsl	Rewritten in Fetch to standard shxadd.
XTheadBb	th.srri, th.srriw, th.rev, th.tst	Rewritten in Fetch to rori, roriw, rev8, and bexti.
XTheadBb	th.ff0, th.ff1	Executed in ALU via 64-bit leading zero count (clz_silice_64).
XTheadBb	th.revw, th.tstnbz	Executed natively in ALU logic blocks.
XTheadBb	th.ext, th.extu	Executed in ALU barrel shifter via variable left/right shift alignment.
XTheadMac	th.mula, th.mulaw,	Hardware integer Multiply-Accumulate

Extension	Instructions	Implementation Mechanism
	th.muls, th.mulsw, th.mulah, th.mulsh	/ Multiply-Subtract using a 130-bit internal product and the third integer register read port ($rs3$/rd).
XTheadCondMov	th.mveqz, th.mvnez	Hardware conditional moves between integer registers based on condition testing against zero.
XTheadMemPair	th.lwd, th.lwud, th.ldd, th.swd, th.sdd	Sequenced across two consecutive dispatch cycles in Fetch.
XTheadMemIdx	th.lrb, th.lurb, th.lrh, th.lurh, th.lrw, th.lurw, th.lrd, th.lurd, th.srb, th.srh, etc.	Evaluated via indexed address calculation in AGU (AGU_I1 + (AGU_I2 << shift)) and micro-sequencing.
XTheadFMemIdx	th.flrw, th.flrd, th.fsrw, th.fsrd, etc.	Floating-point indexed load and store operations routed to LSA.

3. RISC-V Standard Bit Manipulation Extensions (Zba, Zbb, Zbc, Zbs, Zicond)

The main ALU pipeline (cpu_execute_alu) provides dedicated hardware paths for modern bit manipulation extensions:

- **Zba (Address Generation):** Implements add.uw, sh1add, sh2add, sh3add, and shxadd.uw for scaling array indices by 2, 4, or 8.
- **Zbb (Basic Bit Manipulation):**
- Counting: clz, ctz, cpop and their 32-bit *W variants.
- Sign/Zero-Extension: sext.b, sext.h, zext.h.
- Min/Max: min, max, minu, maxu.
- Rotates: rol, ror, rori, rolw, rorw, roriw.
- Logical-with-Negation: andn, orn, xnor.

- Byte Reversal & Bitwise-OR Combining: rev8, orc.b.
- **Zbc / Zbkb (Carryless Multiply / Permutation):** Implements byte and nibble permutation crossbar logic via xperm.b and xperm.n.
- **Zbs (Single-Bit Operations):** Implements bset, bseti, bclr, bclri, binv, binvi, bext, and bexti.
- **Zicond (Conditional Operations):** Implements branchless evaluation via czero.eqz and czero.nez.

4. RISC-V Zfa (Additional Floating-Point) and Half-Precision (Zfh/Zfhmin)

The floating-point pipeline supports Half (FP16), Single (FP32), and Double (FP64) precision natively through a unified 64-bit internal representation:

- **Floating-Point Load Immediate (FLI):** Implements the RISC-V Zfa extension FLI.H, FLI.S, and FLI.D using a 96-entry lookup table (immediate_rom). It loads 32 mathematical constants, powers of 2, minimum normal values, and special IEEE-754 values directly into floating-point registers without memory access.
- **Modular Conversion (FCVTMOD.W.D):** Converts double-precision floats to signed 32-bit integers with 2's complement wrapping semantics (used for JavaScript runtime acceleration).
- **Direct Rounding (FROUND / FROUNDNX):** Implements round-to-integer instructions supporting IEEE rounding modes (RNE, RTZ, RDN, RUP).
- **IEEE 754-2019 Min/Max (FMINM / FMAXM):** Distinguishes between standard RISC-V NaN handling and 2019 minimum/maximum number semantics.
- **NaN-Boxing Validation:** The typeF classifier verifies that Half and Single precision inputs residing in 64-bit registers have their upper unused bits set to all ones; if improper NaN-boxing is detected, the operand is coerced into a canonical quiet NaN (qNAN).

5. Multi-Port Register Architecture & Hazard Detection

- **Register File Read Ports:** Integer registers ($I1, I2, I3$) and floating-point registers ($F1, F2, F3$) provide three independent read ports per cycle. This enables single-cycle issue for ternary instructions:

- Integer Multiply-Accumulate / Conditional Moves (rs1, rs2, and rd/rs3).
- Floating-Point Fused Multiply-Add (rs1 × rs2 ± rs3).
- **Hardware Interlocking:** Stage 3 (cpu_registers) uses 64-bit bitmasks (SR_MASK and RD_MASK) to detect Read-After-Write (RAW) and Write-After-Write (WAW) hazards across the 32 integer and 32 floating-point registers. It checks all downstream pipeline stages simultaneously: AGU, Dispatch, Execute ALU, ALUDivide, CSRFPU, Fastbase, LSA, Commit I, and Commit F. If any in-flight register conflicts with an instruction entering Stage 3, the register stage locks and asserts a back-pressure stall to Stage 2 (Decode).

6. Hardware Performance Counters and Profiling CSRs

The CSR control unit (CSRblock) integrates 40-bit hardware performance and event counters:

- **mcycle / rdcycle (0xC00):** Tracks CPU execution cycles, automatically gated during system DMA transfers (~DMA_ACTIVE) to record CPU execution cycles accurately.
- **time / rdtime (0xC01):** Increments continuously based on the primary system tick input.
- **minstret / rdinstret (0xC02):** Accurately increments based on instructions retired per cycle. It monitors commit acknowledgments from all five execution units, branch completions from the AGU, and store bypass operations (CSR_INC_INSNRET).
- **mhpmevent / mhpmcounter (0x320–0x33F):** Mapped to hardware status lines, reporting direct DMA operational status for runtime profiling.
- **misa (0x301):** Encoded as 0x4001102F, declaring a 64-bit base architecture with extensions A (Atomic), C (Compressed), D (Double-precision float), F (Single-precision float), I (Base integer), M (Multiply/Divide), and non-standard extensions.

7. Cache & Memory Fence Synchronization

- **FENCE.I and CPU_FENCE Execution:** Executed through Stage 4A (AGU). When a FENCE.I instruction clears the pipeline, it asserts CPU_FENCE to trigger parallel sweep invalidation engines in both the L1 instruction cache (iL1cache_write) and the branch predictor table (branch_predict_write), zeroing valid bits across all 2048 L1 blocks and 64 branch predictor entries.

PAWSv3 RISC-V CPU Interrupt Architecture

Interrupt Sources and Vectors

The PAWSv3 CPU supports three primary asynchronous hardware exceptions, each mapping to a distinct interrupt vector:

- **Software Interrupt (ECALL):** Maps to exception vector 3 (IRQ_CAUSE_SOFTWARE) and is triggered by the ECALL instruction.
- **Timer Interrupt:** Maps to exception vector 7 (IRQ_CAUSE_TIMER). This interrupt is triggered when the system's 64-bit main tick counter (RDTIME) matches the value programmed into the IRQ_TIMER comparator, provided the TIMER_VALID flag is set.
- **External / VBLANK Interrupt:** Maps to exception vector 11 (IRQ_CAUSE_VBLANK). This is driven by the system's IRQ_VBLANK hardware signal.

Control and Status Registers (CSRs)

The CSRblock module handles the registers required for interrupt context and delegation:

- **MSTATUS (Machine Status):** Controls global interrupt execution. Bit 3 (MIE - Machine Interrupt Enable) must be high for interrupts to fire, while Bit 7 (MPIE - Machine Previous Interrupt Enable) is used to preserve the prior state during trap handling.
- **MIE and MIP (Machine Interrupt Enable / Pending):** Bit 7 of these registers correlates to the Timer Interrupt, while Bit 11 handles the External/VBLANK interrupt.
- **MCAUSE (Machine Cause):** Stores the active exception code (3, 7, or 11) when the interrupt fires.
- **MEPC (Machine Exception Program Counter):** Automatically captures the Program Counter of the interrupted instruction to allow resumption.
- **MTVEC (Machine Trap-Vector Base-Address):** Holds the base address for the interrupt service routines. If the lowest bit is set (Vectored Mode), the CPU automatically calculates the jump address by shifting the vector number left by two (cause * 4) and adding it to the base address.
- **MSCRATCH:** A 64-bit scratch register provided for the software trap handler to save and restore context without overwriting general-purpose registers.

Interrupt Execution Flow

1. **Triggering:** The CSR unit evaluates incoming hardware triggers alongside the CSR registers. If MSTATUS MIE is enabled and a matching bit is set in both the MIE and MIP registers, the unit asserts the CSR_IRQ_PENDING signal.
2. **Fetch Interception:** The Fetch pipeline stage (cpu_fetch) constantly monitors for pending interrupts. When CSR_IRQ_PENDING is asserted, it waits for the pipeline to clear any currently pending branch instructions. Once clear, it halts normal instruction fetching, enters STATE_FETCH_IRQ_START, and sends a CSR_IRQ_ACK signal to the CSR unit.
3. **Context Saving:** Upon receiving the acknowledgment, the CSR block saves the next scheduled Program Counter into MEPC and sets MCAUSE to the appropriate vector code. It then pushes the MSTATUS MIE bit into MPIE, clears MIE to disable nested interrupts, and replies with IRQ_ACK_ACK.
4. **Vector Jump:** The Fetch stage reads the computed vector address from the CSR unit, updates the Program Counter, and immediately resumes fetching instructions from the trap handler.

Return from Interrupt (MRET)

1. **Execution:** When the interrupt handler issues an MRET instruction, it is identified during the Decode stage and passes down the pipeline to the Address Generation Unit (AGU).
2. **Restoration Signal:** The AGU routes the branch jump address directly to the value stored in CSR_MEPC and asserts the CSR_IRQ_END flag back to the CSR unit.
3. **Completion:** The CSR unit detects IRQ_END, restores the global MIE bit using the value saved in MPIE, clears MCAUSE to 0, and asserts IRQ_END_ACK. The AGU receives this acknowledgment and directs the Fetch unit to safely restart execution at the restored Program Counter.

PAWSlibrary Interrupt Management Functions

The PAWSlibrary provides a suite of C functions to interface directly with the RISC-V Control and Status Registers (CSRs) and memory-mapped I/O to manage hardware exceptions and timers.

Core Interrupt Control

- **IRQ_ON(int IRQ, int MIE)** Enables specific hardware interrupts by setting the corresponding bits in the Machine Interrupt Enable (mie) CSR. If the MIE flag is passed as a non-zero value, it simultaneously enables global interrupts by setting bit 3 of the Machine Status (mstatus) CSR via the csrsi instruction.
- **IRQ_OFF(int IRQ, int MIE)** Disables specific interrupts by clearing the designated bits in the mie CSR. If the MIE flag is non-zero, it globally disables all interrupts by writing 0 to the mstatus CSR.
- **IRQ_ACK(int IRQ)** Acknowledges that an interrupt has been processed by clearing the associated trigger bit in the Machine Interrupt Pending (mip) CSR.

Vector and Trap Management

- **IRQ_VECTOR(void *function)** Configures the execution entry point for interrupt handling. This function writes a C function pointer directly into the Machine Trap-Vector Base-Address (mtvec) CSR.
- **unsigned long IRQ_CAUSE()** Reads the Machine Cause (mcause) CSR and returns the exception code. This allows the interrupt service routine to identify the specific hardware trigger (e.g., Software, Timer, or VBLANK) that invoked the trap.

Hardware Timer Interrupt Configuration

The library provides three methods for scheduling timer-based interrupts, which interface with the memory-mapped IRQ_TIMER hardware comparator:

- **IRQ_SET_TIMER(long pulses)** Schedules an interrupt for a precise future cycle count. It reads the current 64-bit system tick via CSRtime(), adds the requested pulses, and writes the absolute target value to the IRQ_TIMER_COMPARATOR memory-mapped register.
- **IRQ_SET_ADD(unsigned int pulses)** Schedules a timer interrupt by directly passing a 32-bit pulses value to the IRQ_TIMER_ADD I/O register. The hardware automatically calculates the target by adding this value to the current RDTIME and validates the timer.

- **IRQ_SET_TIMER_QUICK(unsigned int divider)** Configures rapid, predefined interrupt intervals using the hardware's internal lookup table. It writes the divider index (0 through 3) to the IRQ_TIMER_NEXT register, instructing the hardware to add a predefined pulse count (50000, 5000, 500, or 50) to the current RDTIME.